PYTHON PROGRAMMING ASSIGNMENT

10 Programs with Problem Statements, Algorithms, Flowcharts, Pseudocode, Source Code, Test Cases and Outputs

Name	B. Naga Santhosh
Roll Number	26000855
Branch	CSM
Section	1

TOPICS

1. University Scholarship Allocation
2. Progressive Electricity Billing
3. Hospital Billing and Patient Classification
4. Employee Payroll and Deduction Analysis
5. Multi-Plan Telecommunications Cost Optimization
6. Railway Fare and Reservation Calculator
7. Retail Billing with Multiple Discount Rules
8. Manufacturing Cost and Profitability Analysis
9. Multi-Currency Travel Budget Analysis
10. Campus Event Financial Planning

Note: Numerical rates and policy rules are illustrative assumptions for academic practice. Replace them with any exact values supplied by your instructor.

1. University Scholarship Allocation

Problem Statement

Develop a program that checks scholarship applicants against aggregate marks, minimum subject marks, attendance, and annual family income. Rank eligible students and use attendance and lower family income to break ties.

Objective

Identify eligible students and display their rank using predefined rules.

Assumptions

• Aggregate marks must be at least 75%.

• The lowest subject mark must be at least 60%.

• Attendance must be at least 75%.

• Annual family income must not exceed Rs. 300,000.

• Tie-breaking order: higher aggregate, higher attendance, then lower family income.

Algorithm

1. Read the number of applicants.

2. For each applicant, read name, aggregate marks, lowest subject mark, attendance, and annual family income.

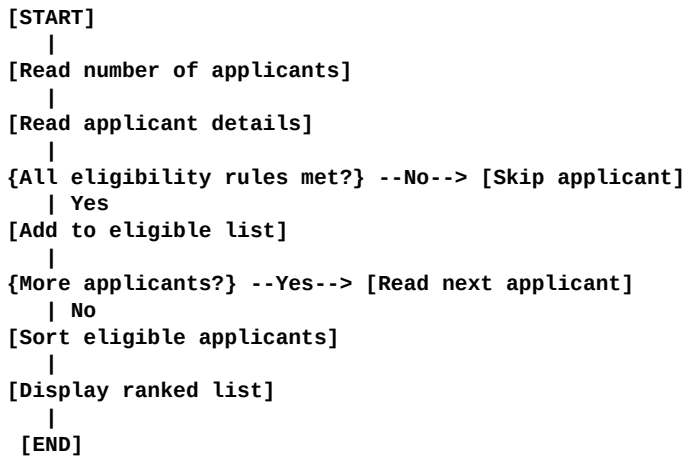
3. Check all four eligibility conditions.

4. Store eligible applicants in a list.

5. Sort by aggregate descending, attendance descending, and income ascending.

6. Display eligible applicants with rank, or show that no applicant qualifies.

Flowchart



Pseudocode

```
BEGIN
  INPUT n
  eligible = empty list
  REPEAT n times
    INPUT name, aggregate, subject_min, attendance, income
    IF aggregate >= 75 AND subject_min >= 60 AND
       attendance >= 75 AND income <= 300000 THEN
       ADD applicant to eligible
    ENDIF
  END REPEAT
  SORT eligible BY aggregate DESC, attendance DESC, income ASC
  DISPLAY eligible with rank
END
```

Source Code

```
students = []
n = int(input("Number of students: "))
```

```
for i in range(n):
    print("\nStudent", i + 1)
    name = input("Name: ")
    aggregate = float(input("Aggregate marks: "))
    subject = float(input("Lowest subject mark: "))
    attendance = float(input("Attendance percentage: "))
    income = float(input("Annual family income: "))

    if aggregate >= 75 and subject >= 60 and attendance >= 75 and income <= 300000:
        students.append((name, aggregate, attendance, income))

students.sort(key=lambda s: (-s[1], -s[2], s[3]))

print("\nEligible Scholarship Students")
if not students:
    print("No eligible students")
else:
    for rank, s in enumerate(students, start=1):
        print(rank, s[0], "Marks:", s[1],
              "Attendance:", s[2], "Income:", s[3])
```

Test Case

Input: 2 students — Ravi, aggregate 85, lowest subject 70, attendance 90, income 200000; Sita, aggregate 92, lowest subject 75, attendance 88, income 250000.

Expected Output

```
Eligible Scholarship Students
1 Sita Marks: 92.0 Attendance: 88.0 Income: 250000.0
2 Ravi Marks: 85.0 Attendance: 90.0 Income: 200000.0
```

2. Progressive Electricity Billing

Problem Statement

Calculate a household's electricity bill using progressive consumption slabs, a fixed charge, a surcharge for high bills, and a rebate for low consumption. Compare two months and flag increases greater than 30%.

Objective

Compute accurate monthly bills and identify unusual increases in consumption cost.

Assumptions

• First 100 units cost Rs. 2 per unit.

• Units 101–200 cost Rs. 3 per unit; units above 200 cost Rs. 5 per unit.

• Fixed charge is Rs. 50.

• Add a 5% surcharge if the bill before surcharge exceeds Rs. 500.

• Apply a Rs. 20 rebate when consumption is 50 units or less.

• An increase greater than 30% is considered unusual.

Algorithm

1. Read units consumed in two months.

2. For each month, calculate energy charges using the progressive slabs.

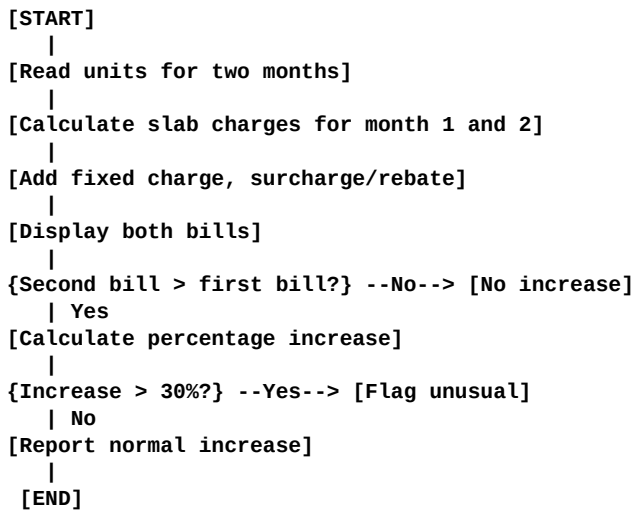
3. Add the fixed charge.

4. Apply the surcharge and low-consumption rebate when applicable.

5. Display both bills.

6. Calculate percentage increase if the second bill is higher and flag increases above 30%.

Flowchart



Pseudocode

```
BEGIN
  INPUT units1, units2
  FOR each units value
    CALCULATE energy using progressive slabs
    bill = energy + 50
    IF bill > 500 THEN bill = bill * 1.05
    IF units <= 50 THEN bill = bill - 20
  END FOR
  DISPLAY bill1, bill2
  IF bill2 > bill1 THEN
    increase = (bill2 - bill1) / bill1 * 100
    IF increase > 30 THEN DISPLAY unusual increase
  ENDIF
END
```

Source Code

```
def calculate_bill(units):
    if units <= 100:
        energy = units * 2
    elif units <= 200:
        energy = 100 * 2 + (units - 100) * 3
    else:
        energy = 100 * 2 + 100 * 3 + (units - 200) * 5

    bill = energy + 50
    if bill > 500:
        bill += bill * 0.05
    if units <= 50:
        bill -= 20
    return bill

month1 = float(input("Units in month 1: "))
month2 = float(input("Units in month 2: "))
bill1 = calculate_bill(month1)
bill2 = calculate_bill(month2)

print("Month 1 bill: Rs.", round(bill1, 2))
print("Month 2 bill: Rs.", round(bill2, 2))

if bill2 > bill1:
    increase = (bill2 - bill1) / bill1 * 100
    print("Increase:", round(increase, 2), "%")
    if increase > 30:
        print("Unusual increase detected")
    else:
        print("Normal increase")
else:
    print("No increase in the latest bill")
```

Test Case

Input: Month 1 = 100 units; Month 2 = 250 units.

Expected Output

```
Month 1 bill: Rs. 250.0
Month 2 bill: Rs. 892.5
Increase: 257.0 %
Unusual increase detected
```

3. Hospital Billing and Patient Classification

Problem Statement

Calculate a patient's consultation, diagnostic, and treatment charges. Apply a concession when the bill reaches a specified threshold, validate the payment, and classify the payment status.

Objective

Produce a final hospital bill, balance due, and payment classification.

Assumptions

• All charges must be non-negative.

• A 10% concession applies when total charges are at least Rs. 5,000.

• Payment must be between zero and the final bill.

• Status is Fully Paid if balance is zero, Partially Paid if some amount is paid, otherwise Unpaid.

Algorithm

1. Read patient name and three charge amounts.

2. Add the charges to find the total.

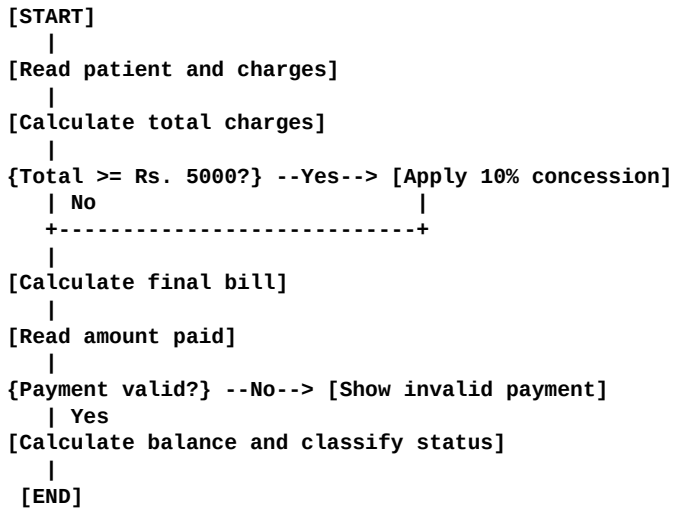
3. Apply a 10% concession if the total is at least Rs. 5,000.

4. Calculate the final bill.

5. Read the amount paid and validate it.

6. Display the bill, concession, balance, and payment status.

Flowchart



Pseudocode

```
BEGIN
  INPUT name, consultation, diagnostic, treatment
  total = consultation + diagnostic + treatment
  IF total >= 5000 THEN concession = total * 0.10
  ELSE concession = 0
  final_bill = total - concession
  INPUT paid
  IF paid < 0 OR paid > final_bill THEN DISPLAY invalid payment
  ELSE
    balance = final_bill - paid
    IF balance = 0 THEN status = Fully Paid
    ELSE IF paid > 0 THEN status = Partially Paid
    ELSE status = Unpaid
    DISPLAY bill, balance, status
  ENDIF
END
```

Source Code

```
name = input("Patient name: ")
consultation = float(input("Consultation charge: "))
diagnostic = float(input("Diagnostic charge: "))
treatment = float(input("Treatment charge: "))
paid = float(input("Amount paid: "))

if min(consultation, diagnostic, treatment) < 0:
    print("Invalid charges")
else:
    total = consultation + diagnostic + treatment
    concession = total * 0.10 if total >= 5000 else 0
    final_bill = total - concession

    if paid < 0 or paid > final_bill:
        print("Invalid payment amount")
    else:
        balance = final_bill - paid
        print("\nPatient:", name)
        print("Total charges: Rs.", round(total, 2))
        print("Concession: Rs.", round(concession, 2))
        print("Final bill: Rs.", round(final_bill, 2))
        print("Balance: Rs.", round(balance, 2))

        if balance == 0:
            print("Classification: Fully Paid")
        elif paid > 0:
            print("Classification: Partially Paid")
        else:
            print("Classification: Unpaid")
```

Test Case

Patient Ravi; consultation Rs. 500; diagnostics Rs. 1,500; treatment Rs. 4,000; amount paid Rs. 3,000.

Expected Output

```
Patient: Ravi
Total charges: Rs. 6000.0
Concession: Rs. 600.0
Final bill: Rs. 5400.0
Balance: Rs. 2400.0
Classification: Partially Paid
```

4. Employee Payroll and Deduction Analysis

Problem Statement

Calculate gross salary from basic pay, allowances, overtime, and incentives. Apply progressive tax deductions and print a salary statement.

Objective

Determine the employee's gross salary, tax deduction, and net salary.

Assumptions

• Overtime is paid at Rs. 200 per hour.

• Gross salary up to Rs. 20,000 is taxed at 5%.

• The portion from Rs. 20,001 to Rs. 40,000 is taxed at 10%.

• The portion above Rs. 40,000 is taxed at 20%.

• These are demonstration rates, not official tax advice.

Algorithm

1. Read basic salary, allowances, overtime hours, and incentive.

2. Calculate overtime pay at Rs. 200 per hour.

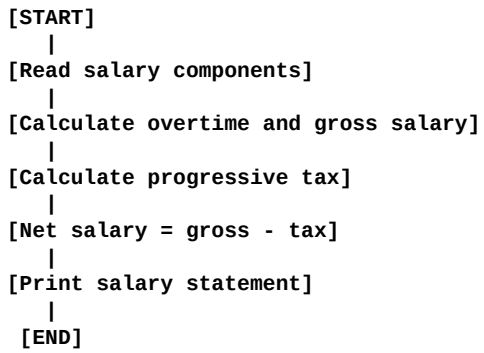
3. Add all earnings to calculate gross salary.

4. Calculate tax progressively across the salary brackets.

5. Subtract tax from gross salary.

6. Print the salary statement.

Flowchart



Pseudocode

```
BEGIN
  INPUT basic, allowances, overtime_hours, incentive
  overtime = overtime_hours * 200
  gross = basic + allowances + overtime + incentive
  IF gross <= 20000 THEN tax = gross * 0.05
  ELSE IF gross <= 40000 THEN
    tax = 20000*0.05 + (gross-20000)*0.10
  ELSE
    tax = 20000*0.05 + 20000*0.10 + (gross-40000)*0.20
  net = gross - tax
  DISPLAY earnings, gross, tax, net
END
```

Source Code

```
basic = float(input("Basic salary: "))
allowances = float(input("Allowances: "))
hours = float(input("Overtime hours: "))
incentive = float(input("Incentive: "))

overtime = hours * 200
gross = basic + allowances + overtime + incentive
```

```
if gross <= 20000:
    tax = gross * 0.05
elif gross <= 40000:
    tax = 20000 * 0.05 + (gross - 20000) * 0.10
else:
    tax = (20000 * 0.05 + 20000 * 0.10
           + (gross - 40000) * 0.20)

net = gross - tax

print("\n--- SALARY STATEMENT ---")
print("Basic salary: Rs.", round(basic, 2))
print("Allowances: Rs.", round(allowances, 2))
print("Overtime pay: Rs.", round(overtime, 2))
print("Incentive: Rs.", round(incentive, 2))
print("Gross salary: Rs.", round(gross, 2))
print("Tax deduction: Rs.", round(tax, 2))
print("Net salary: Rs.", round(net, 2))
```

Test Case

Basic Rs. 25,000; allowances Rs. 5,000; overtime 10 hours; incentive Rs. 2,000.

Expected Output

```
--- SALARY STATEMENT ---
Basic salary: Rs. 25000.0
Allowances: Rs. 5000.0
Overtime pay: Rs. 2000.0
Incentive: Rs. 2000.0
Gross salary: Rs. 34000.0
Tax deduction: Rs. 2400.0
Net salary: Rs. 31600.0
```

5. Multi-Plan Telecommunications Cost Optimization

Problem Statement

Compare mobile plans based on data usage, required validity, and budget. Include excess-data charges and select the lowest-cost eligible plan.

Objective

Recommend the most economical plan that satisfies the user's requirements.

Assumptions

• Basic: Rs. 199, 2 GB, 28 days, Rs. 50 per extra GB.

• Smart: Rs. 299, 10 GB, 28 days, Rs. 20 per extra GB.

• Premium: Rs. 399, 20 GB, 56 days, Rs. 10 per extra GB.

• Extra data is charged proportionally per GB.

• A plan is eligible if validity and total cost meet the user's needs and budget.

Algorithm

1. Read required data, validity, and budget.

2. Store each plan's price, data limit, validity, and excess rate.

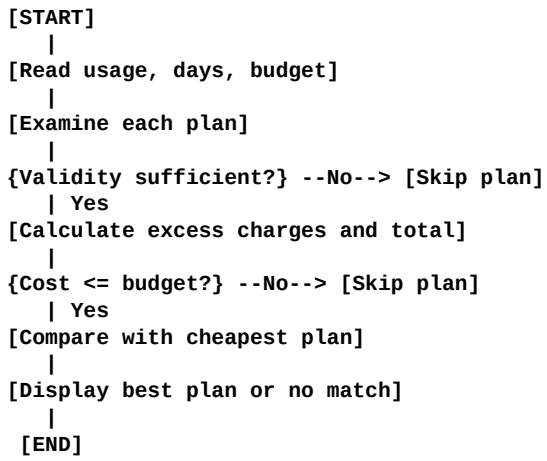
3. Calculate extra data and total cost for each plan with sufficient validity.

4. Keep plans within budget.

5. Choose the eligible plan with the lowest total cost.

6. Display the best plan or report no eligible plan.

Flowchart



Pseudocode

```
BEGIN
  INPUT usage, required_days, budget
  plans = predefined plan list
  best_cost = infinity
  FOR each plan
    IF plan.validity >= required_days THEN
      excess = MAX(0, usage - plan.data)
      cost = plan.price + excess * plan.extra_rate
      IF cost <= budget AND cost < best_cost THEN
        best_plan = plan.name
        best_cost = cost
      ENDIF
    ENDIF
  END FOR
  DISPLAY best_plan and best_cost, or no eligible plan
END
```

Source Code

```
usage = float(input("Required data in GB: "))
days = int(input("Required validity in days: "))
budget = float(input("Budget in Rs.: "))

plans = [
    ("Basic", 199, 2, 28, 50),
    ("Smart", 299, 10, 28, 20),
    ("Premium", 399, 20, 56, 10)
]

best_plan = None
best_cost = float("inf")

for name, price, data, validity, extra_rate in plans:
    if validity >= days:
        excess = max(0, usage - data)
        cost = price + excess * extra_rate
        if cost <= budget and cost < best_cost:
            best_cost = cost
            best_plan = name

if best_plan is None:
    print("No eligible plan found")
else:
    print("Best plan:", best_plan)
    print("Total cost: Rs.", round(best_cost, 2))
```

Test Case

Required data 8 GB; validity 28 days; budget Rs. 350.

Expected Output

```
Best plan: Smart
Total cost: Rs. 299
```

6. Railway Fare and Reservation Calculator

Problem Statement

Calculate railway fares for multiple passengers based on distance, travel class, age concessions, reservation charges, and cancellation penalties.

Objective

Calculate each passenger's fare and, when applicable, the cancellation refund.

Assumptions

• Sleeper fare is Rs. 1.50 per kilometre; AC fare is Rs. 3.00 per kilometre.

• Passengers under age 5 travel free in this simplified example.

• Passengers aged 60 or older receive a 20% fare concession.

• Reservation charge is Rs. 50 for a non-free ticket.

• Cancellation penalty is Rs. 100 or the ticket fare, whichever is lower.

• These are classroom demonstration rules, not current railway tariffs.

Algorithm

1. Read the number of passengers.

2. For each passenger, read name, distance, age, and class.

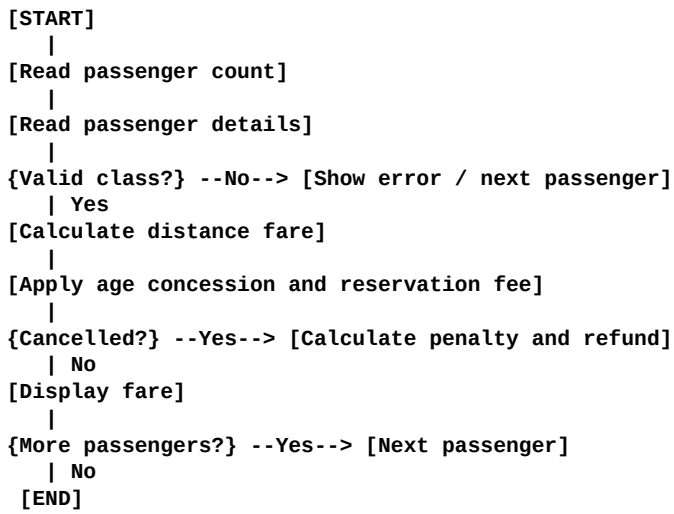
3. Select the rate according to class.

4. Calculate base fare and apply age concession.

5. Add reservation charge where applicable.

6. Ask whether the ticket is cancelled; if yes, calculate penalty and refund.

Flowchart



Pseudocode

```
BEGIN
  INPUT n
  REPEAT n times
    INPUT name, distance, age, class
    SELECT rate based on class
    fare = distance * rate
    IF age < 5 THEN fare = 0
    ELSE IF age >= 60 THEN fare = fare * 0.80
    IF fare > 0 THEN fare = fare + 50
    DISPLAY fare
    INPUT cancel
    IF cancel = yes THEN
      penalty = MIN(fare, 100)
      refund = fare - penalty
```

```
      DISPLAY penalty, refund
    ENDIF
  END REPEAT
END
```

Source Code

```
n = int(input("Number of passengers: "))

for i in range(n):
    print("\nPassenger", i + 1)
    name = input("Name: ")
    distance = float(input("Distance in km: "))
    age = int(input("Age: "))
    travel_class = input("Class (Sleeper/AC): ").lower()

    if travel_class == "sleeper":
        rate = 1.5
    elif travel_class == "ac":
        rate = 3.0
    else:
        print("Invalid class")
        continue

    fare = distance * rate
    if age < 5:
        fare = 0
    elif age >= 60:
        fare *= 0.80

    if fare > 0:
        fare += 50

    print("Passenger:", name)
    print("Ticket fare: Rs.", round(fare, 2))
    cancel = input("Cancel ticket? (yes/no): ").lower()

    if cancel == "yes":
        penalty = min(fare, 100)
        refund = fare - penalty
        print("Cancellation penalty: Rs.", penalty)
        print("Refund: Rs.", round(refund, 2))
```

Test Case

One passenger: Ravi; distance 200 km; age 30; Sleeper class; cancel = yes.

Expected Output

```
Passenger: Ravi
Ticket fare: Rs. 350.0
Cancellation penalty: Rs. 100
Refund: Rs. 250.0
```

7. Retail Billing with Multiple Discount Rules

Problem Statement

Generate an itemized retail invoice for multiple products, applying category-specific taxes and a threshold-based discount subject to a maximum limit.

Objective

Calculate subtotal, taxes, discount, and final amount payable.

Assumptions

• Food tax is 5%, electronics tax is 18%, and other items tax is 12%.

• A 10% discount applies when the pre-tax subtotal is at least Rs. 2,000.

• The maximum discount is Rs. 500.

• Each product has a non-negative price and a positive quantity.

Algorithm

1. Read the number of products.

2. For each product, read its name, category, unit price, and quantity.

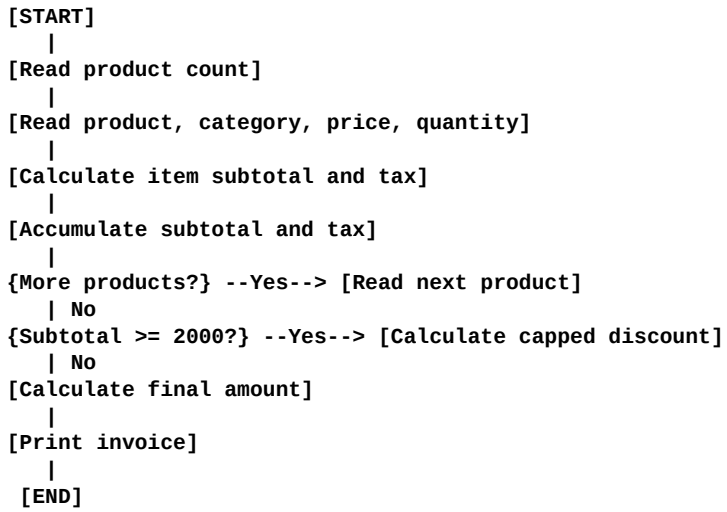
3. Calculate item amount and category-specific tax.

4. Add all item amounts and taxes.

5. Apply a 10% discount if subtotal is at least Rs. 2,000, capped at Rs. 500.

6. Display the invoice totals.

Flowchart



Pseudocode

```
BEGIN
  INPUT n
  subtotal = 0; tax_total = 0
  REPEAT n times
    INPUT name, category, price, quantity
    amount = price * quantity
    SELECT tax rate by category
    tax = amount * tax_rate
    subtotal = subtotal + amount
    tax_total = tax_total + tax
  END REPEAT
  IF subtotal >= 2000 THEN discount = MIN(subtotal*0.10, 500)
  ELSE discount = 0
  total = subtotal + tax_total - discount
  DISPLAY subtotal, tax_total, discount, total
END
```

Source Code

```
n = int(input("Number of products: "))
subtotal = 0
tax_total = 0

for i in range(n):
    print("\nProduct", i + 1)
    name = input("Product name: ")
    category = input("Category (food/electronics/other): ").lower()
    price = float(input("Price: "))
    quantity = int(input("Quantity: "))

    if price < 0 or quantity <= 0:
        print("Invalid price or quantity")
        continue

    amount = price * quantity
    if category == "food":
        tax_rate = 0.05
    elif category == "electronics":
        tax_rate = 0.18
    else:
        tax_rate = 0.12

    tax = amount * tax_rate
    subtotal += amount
    tax_total += tax
    print(name, "Amount:", round(amount, 2),
          "Tax:", round(tax, 2))

discount = min(subtotal * 0.10, 500) if subtotal >= 2000 else 0
total = subtotal + tax_total - discount

print("\n--- FINAL INVOICE ---")
print("Subtotal: Rs.", round(subtotal, 2))
print("Total tax: Rs.", round(tax_total, 2))
print("Discount: Rs.", round(discount, 2))
print("Final amount: Rs.", round(total, 2))
```

Test Case

Rice: food, price Rs. 1,000, quantity 1; Headphones: electronics, price Rs. 1,500, quantity 1.

Expected Output

```
Rice Amount: 1000.0 Tax: 50.0
Headphones Amount: 1500.0 Tax: 270.0
--- FINAL INVOICE ---
Subtotal: Rs. 2500.0
Total tax: Rs. 320.0
Discount: Rs. 250.0
Final amount: Rs. 2570.0
```

8. Manufacturing Cost and Profitability Analysis

Problem Statement

Calculate material cost, wastage, labour, machine expenses, sales revenue, and profit. Check whether the required profit target is achieved.

Objective

Assess production cost and profitability using user-provided values.

Assumptions

• Wastage is calculated as a percentage of total material cost.

• Labour and machine costs are total batch costs.

• All products produced are assumed to be sold at the entered selling price.

• Profit equals revenue minus total production cost.

Algorithm

1. Read quantity, material cost per unit, wastage percentage, labour cost, machine cost, selling price, and profit target.

2. Calculate material cost and wastage cost.

3. Add material, wastage, labour, and machine costs.

4. Calculate sales revenue.

5. Subtract total cost from revenue to get profit.

6. Compare profit against the target.

Flowchart

```
[START]
   |
[Read production and cost data]
   |
[Calculate material cost and wastage]
   |
[Calculate total production cost]
   |
[Calculate sales revenue and profit]
   |
{Profit >= target?} --Yes--> [Target achieved]
   | No
[Target not achieved]
   |
 [END]
```

Pseudocode

```
BEGIN
  INPUT quantity, material_unit, wastage_percent, labour, machine, selling_price, target
  material_cost = quantity * material_unit
  wastage = material_cost * wastage_percent / 100
  total_cost = material_cost + wastage + labour + machine
  revenue = quantity * selling_price
  profit = revenue - total_cost
  DISPLAY total_cost, revenue, profit
  IF profit >= target THEN DISPLAY target achieved
  ELSE DISPLAY target not achieved
END
```

Source Code

```
quantity = int(input("Production quantity: "))
material = float(input("Material cost per unit: "))
wastage_percent = float(input("Wastage percentage: "))
labour = float(input("Total labour cost: "))
machine = float(input("Machine cost: "))
selling_price = float(input("Selling price per unit: "))
target = float(input("Required profit: "))

material_cost = quantity * material
```

```
wastage = material_cost * wastage_percent / 100
total_cost = material_cost + wastage + labour + machine
revenue = quantity * selling_price
profit = revenue - total_cost

print("\nTotal cost: Rs.", round(total_cost, 2))
print("Sales revenue: Rs.", round(revenue, 2))
print("Profit/Loss: Rs.", round(profit, 2))

if profit >= target:
    print("Profit target achieved")
else:
    print("Profit target not achieved")
```

Test Case

Quantity 100; material Rs. 50/unit; wastage 10%; labour Rs. 1,000; machine Rs. 500; selling price Rs. 80/unit; target profit Rs. 2,000.

Expected Output

```
Total cost: Rs. 7000.0
Sales revenue: Rs. 8000.0
Profit/Loss: Rs. 1000.0
Profit target not achieved
```

9. Multi-Currency Travel Budget Analysis

Problem Statement

Convert travel expenses using supplied exchange rates, add currency conversion fees, compare total spending with the budget, and identify the largest expense category.

Objective

Summarize foreign-currency travel spending in Indian rupees and determine budget status.

Assumptions

• Example exchange rates: Hotel currency 1 unit = Rs. 83; Food currency = Rs. 90; Transport currency = Rs. 105.

• A 2% conversion fee applies to each converted expense.

• Exchange rates are illustrative inputs, not live rates.

Algorithm

1. Read the travel budget in rupees.

2. Read hotel, food, and transport expenses in the corresponding foreign currency.

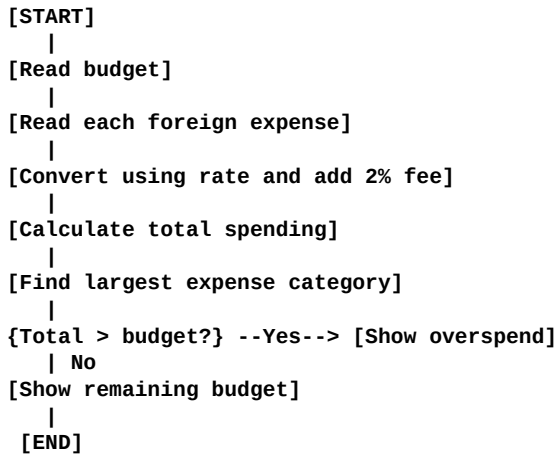
3. Convert each expense using its example rate.

4. Add a 2% fee to each converted amount.

5. Calculate total spending and remaining budget or overspend.

6. Identify the category with the highest final converted cost.

Flowchart



Pseudocode

```
BEGIN
  INPUT budget
  rates = {Hotel:83, Food:90, Transport:105}
  FOR each category and rate
    INPUT foreign_amount
    converted = foreign_amount * rate
    final_amount = converted * 1.02
    STORE final_amount
  END FOR
  total = SUM(expenses)
  largest = category with maximum expense
  DISPLAY expenses, total, largest
  IF total > budget DISPLAY overspend
  ELSE DISPLAY remaining budget
END
```

Source Code

```
budget = float(input("Travel budget in Rs.: "))

rates = {"Hotel": 83, "Food": 90, "Transport": 105}
expenses = {}
```

```
total = 0

for category, rate in rates.items():
    amount = float(input(category + " expense in foreign currency: "))
    converted = amount * rate
    final_amount = converted * 1.02
    expenses[category] = final_amount
    total += final_amount

print("\n--- TRAVEL EXPENSE REPORT ---")
for category, amount in expenses.items():
    print(category, ": Rs.", round(amount, 2))

largest = max(expenses, key=expenses.get)
print("Total spending: Rs.", round(total, 2))
print("Largest expense:", largest)

if total > budget:
    print("Budget exceeded by Rs.", round(total - budget, 2))
else:
    print("Remaining budget: Rs.", round(budget - total, 2))
```

Test Case

Budget Rs. 20,000; hotel 100 units at Rs. 83; food 50 units at Rs. 90; transport 20 units at Rs. 105.

Expected Output

```
Hotel : Rs. 8466.0
Food : Rs. 4590.0
Transport : Rs. 2142.0
Total spending: Rs. 15198.0
Largest expense: Hotel
Remaining budget: Rs. 4802.0
```

10. Campus Event Financial Planning

Problem Statement

Calculate campus event costs for venue, equipment, catering, transport, and certificates. Evaluate sponsorship shortfall, participant contribution, and minimum registrations to break even.

Objective

Create an event budget and calculate how many registrations are needed to cover costs after sponsorship.

Assumptions

• Venue, equipment, and transport are fixed costs.

• Catering and certificates are variable costs per participant.

• Registration fee is collected from each participant.

• Sponsorship reduces the fixed costs to be covered by registration revenue.

• Break-even registrations = ceiling((fixed costs - sponsorship) / (registration fee - variable cost per participant)), when the denominator is positive.

Algorithm

1. Read fixed costs, per-person catering and certificate costs, sponsorship, expected participants, and registration fee.

2. Calculate fixed costs and variable costs for expected participants.

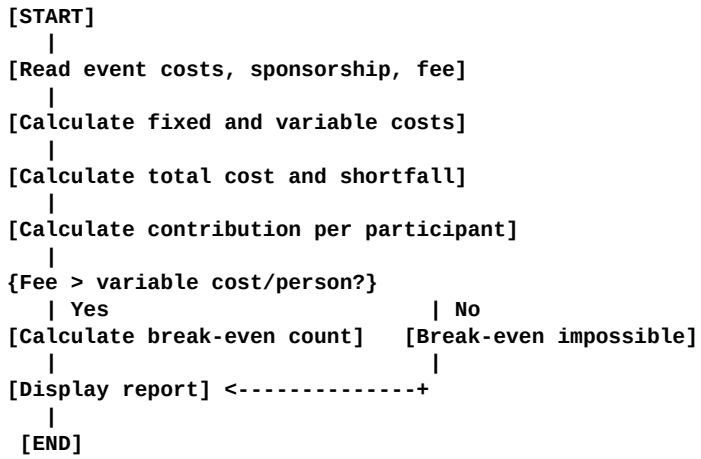
3. Calculate total cost and funding shortfall after sponsorship.

4. Calculate the per-participant contribution needed to cover the shortfall.

5. Calculate minimum registrations using fixed costs, sponsorship, and contribution per participant.

6. Display the budget report.

Flowchart



Pseudocode

```
BEGIN
  INPUT venue, equipment, catering_each, transport, certificate_each
  INPUT sponsorship, participants, registration_fee
  fixed_cost = venue + equipment + transport
  variable_each = catering_each + certificate_each
  total_cost = fixed_cost + participants * variable_each
  shortfall = MAX(0, total_cost - sponsorship)
  contribution = MAX(0, shortfall / participants) if participants > 0
  IF registration_fee > variable_each THEN
    minimum = MAX(0, CEILING((fixed_cost - sponsorship) / (registration_fee - variable_each)))
  ELSE
    minimum = impossible
  DISPLAY total_cost, shortfall, contribution, minimum
END
```

Source Code

```
import math

venue = float(input("Venue cost: "))
equipment = float(input("Equipment cost: "))
catering = float(input("Catering cost per person: "))
transport = float(input("Transport cost: "))
certificate = float(input("Certificate cost per person: "))
sponsorship = float(input("Sponsorship amount: "))
participants = int(input("Expected participants: "))
fee = float(input("Registration fee per person: "))

fixed_cost = venue + equipment + transport
variable_each = catering + certificate
total_cost = fixed_cost + participants * variable_each
shortfall = max(0, total_cost - sponsorship)

contribution = shortfall / participants if participants > 0 else 0

print("\n--- EVENT BUDGET ---")
print("Total cost: Rs.", round(total_cost, 2))
print("Sponsorship: Rs.", round(sponsorship, 2))
print("Funding shortfall: Rs.", round(shortfall, 2))
print("Contribution per participant: Rs.", round(contribution, 2))

if fee > variable_each:
    minimum = max(0, math.ceil(
        (fixed_cost - sponsorship) / (fee - variable_each)
    ))
    print("Minimum registrations to break even:", minimum)
else:
    print("Break-even is not possible at this registration fee")
```

Test Case

Venue Rs. 10,000; equipment Rs. 5,000; catering Rs. 200/person; transport Rs. 3,000; certificate Rs. 50/person; sponsorship Rs. 10,000; expected participants 100; registration fee Rs. 300.

Expected Output

```
--- EVENT BUDGET ---
Total cost: Rs. 40000.0
Sponsorship: Rs. 10000.0
Funding shortfall: Rs. 30000.0
Contribution per participant: Rs. 300.0
Minimum registrations to break even: 160
```